BÁO CÁO TIẾN ĐỘ HỌC TẬP

Linux, Virtualization

→ Container & Kubernetes

Tổng kết kiến thức Tuần 1 và Tuần 2

Linux | Network | Storage | Permission | Hypervisor | Container | Kubernetes

Người báo cáo: ..... · Ngày: .....

TỔNG QUAN

Tiến độ 2 tuần đầu

2	8	6	2
tuần đã hoàn thành	mảng kiến thức	mảng hạ tầng nền tảng	mảng Container & K8s

Tuần 1 – Linux, Virtualization và hạ tầng nền tảng

1. Linux doanh nghiệp: quản lý CPU, RAM, disk, process
2. Network: IP, routing, DNS, kết nối giữa các VM
3. Storage: partition, LVM, filesystem, mount
4. User, group và permission
5. Service (systemd) và firewall
6. Hypervisor và vòng đời VM

Tuần 2 – Container và Kubernetes Foundation

7. Container: image, registry, volume, network, resource limit
8. Kiến trúc Kubernetes và vai trò từng thành phần

Mạch kiến thức: hiểu máy chủ Linux → ảo hóa thành nhiều VM → đóng gói ứng dụng thành container → điều phối container ở quy mô cluster bằng Kubernetes.

TUẦN 1

Linux, Virtualization và hạ tầng nền tảng

Tài nguyên máy chủ | Network | Storage & LVM | User & Permission | systemd & Firewall | Hypervisor & VM

TUẦN 1 · CHỦ ĐỀ 1

Linux doanh nghiệp: quản lý tài nguyên máy chủ

CPU

- **Load average** (1/5/15 phút): số tiến trình đang chạy hoặc chờ CPU. So với số core (`nproc`): load > số core là quá tải
- `%us` user · `%sy` kernel · `%wa` chờ I/O · `%st` bị hypervisor lấy (steal)
- `%wa` cao → nghẽn disk, không phải thiếu CPU

RAM

- Nhìn cột **available**, không nhìn cột *free*: Linux dùng RAM trống làm **buff/cache** và trả lại khi cần
- **Swap** tăng liên tục → thiếu RAM thật, hệ thống chậm
- Hết RAM → **OOM killer** chọn và kill tiến trình (xem bằng `dmesg`)

Disk

- `df -h` dung lượng theo filesystem · `du -sh` theo thư mục
- `df -i` : hết **inode** cũng không ghi được file dù còn dung lượng
- `iostat -x` / `iotop` : tìm tiến trình đọc ghi nhiều

Process

- Mỗi tiến trình có **PID**, **PPID**, trạng thái `R` chạy · `S` ngủ · `D` chờ I/O · `Z` zombie
- Ưu tiên: `nice` từ −20 (cao nhất) đến 19 · `renice` đổi khi đang chạy
- Dừng: `kill -15` (SIGTERM, dừng êm) trước, `kill -9` (SIGKILL) khi bị treo

TUẦN 1 · CHỦ ĐỀ 1

Quy trình kiểm tra tài nguyên khi máy chủ chậm

```
uptime                      # load average 1/5/15 phút
nproc                       # số core để so sánh với load
top -o %CPU                 # tiến trình ngốn CPU (htop: giao diện dễ đọc)
free -h                     # RAM: total / used / available / swap
vmstat 1 5                  # r (hàng đợi CPU), si/so (swap), wa (I/O)
df -h ; df -i               # dung lượng và inode
du -sh /var/* | sort -h     # thư mục chiếm nhiều dung lượng
ps aux --sort=-%mem | head  # top tiến trình dùng RAM
renice -n 10 -p <PID>       # hạ ưu tiên tiến trình nền
kill -15 <PID>              # dừng tiến trình
```

Phân bổ tài nguyên cho ứng dụng

- Giới hạn bằng **systemd**: `CPUQuota=50%` , `MemoryMax=1G` (dựa trên cgroups)
- Giới hạn theo user: `ulimit` / `/etc/security/limits.conf` (số file mở, số process)
- Với VM: điều chỉnh vCPU / RAM trên hypervisor

Nguyên tắc: đo trước rồi mới kết luận. Chậm có thể do CPU, RAM, disk I/O hoặc mạng.

🎯 **Đầu ra:** Kiểm tra và phân bổ được tài nguyên; giải thích được cho người khác

TUẦN 1 · CHỦ ĐỀ 2

Network: IP, subnet và routing

IP Địa chỉ IP và subnet

- IPv4 gồm 32 bit, viết dạng `192.168.10.15/24`
- **/24** → mask `255.255.255.0` → 256 địa chỉ, **254** dùng được (trừ địa chỉ mạng và broadcast)
- Dải private (RFC 1918): `10.0.0.0/8` · `172.16.0.0/12` · `192.168.0.0/16`
- **Gateway**: router để đi ra ngoài subnet

Routing

- Cùng subnet → gửi thẳng (dùng ARP tìm MAC)
- Khác subnet → gửi tới gateway theo bảng định tuyến
- **Default route** `0.0.0.0/0` : đường đi khi không khớp route nào

```
ip -br a                          # IP từng interface
ip route                          # bảng định tuyến
ip route add 10.20.0.0/24 via 192.168.10.1
ss -tulpn                         # port đang lắng nghe
nmcli con mod eth0 ipv4.addresses 192.168.10.15/24 \
  ipv4.gateway 192.168.10.1 ipv4.method manual
```

CIDR	Netmask	Số host dùng được
/30	255.255.255.252	2
/24	255.255.255.0	254
/16	255.255.0.0	65.534

TUẦN 1 · CHỦ ĐỀ 2

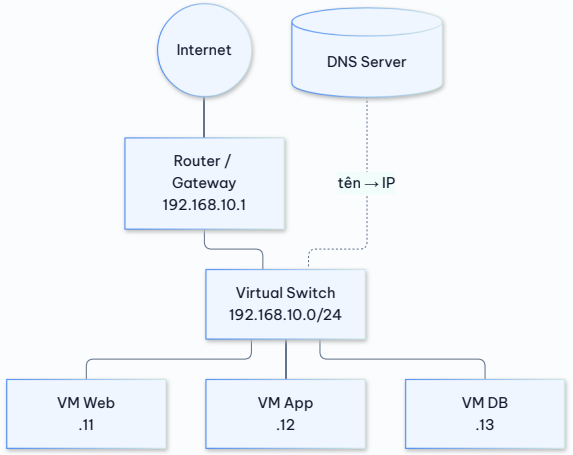
DNS và kết nối mạng giữa các VM

DNS phân giải tên miền

1. Kiểm tra `/etc/hosts` (thứ tự cấu hình trong `/etc/nsswitch.conf`)
2. Hỏi DNS resolver trong `/etc/resolv.conf`
3. Resolver hỏi lần lượt: Root → TLD (`.com` , `.vn`) → Authoritative

Bản ghi chính: **A** (IPv4) · **AAAA** (IPv6) · **CNAME** (bí danh) · **MX** (mail) · **PTR** (IP → tên)

Chế độ mạng VM	VM ↔ VM	VM ↔ Host	Ra Internet
NAT	Tùy	Hạn chế	✅
Bridged	✅	✅	✅
Host-only	✅	✅	❌
Internal	✅	❌	❌

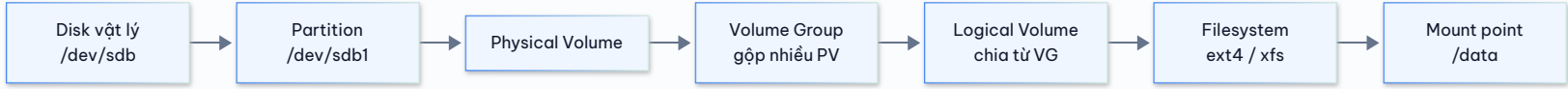


Khi không kết nối được, kiểm tra từ dưới lên: `ip a` có IP chưa → `ping` gateway → `ping` IP đích → `dig` tên miền → `nc -zv host port` kiểm tra port → firewall

🎯 **Đầu ra:** Vẽ và giải thích được luồng kết nối mạng giữa các VM trong Lab

TUẦN 1 · CHỦ ĐỀ 3

Storage: partition, LVM và filesystem



Partition

- **MBR**: tối đa 4 primary, disk ≤ 2 TB
- **GPT**: tới 128 partition, disk lớn, dùng với UEFI
- Công cụ: `fdisk`, `gdisk`, `parted`

LVM

- Gộp nhiều disk thành một **VG**
- **Mở rộng LV khi đang chạy**, không cần dừng dịch vụ
- Hỗ trợ snapshot LV
- Tách ứng dụng khỏi disk vật lý

Filesystem

	ext4	xfs
Mở rộng	`resize2fs`	`xfs_growfs`
Thu nhỏ	Được	**Không**
Mặc định	Ubuntu	RHEL

TUẦN 1 · CHỦ ĐỀ 3

Thực hành: thêm disk mới, tạo LVM và mount

```
lsblk                                  # 1. xem disk mới
parted /dev/sdb mklabel gpt            # 2. tạo bảng phân vùng
parted /dev/sdb mkpart primary 0% 100%
pvcreate /dev/sdb1                     # 3. tạo PV
vgcreate vg_data /dev/sdb1             # 4. tạo VG
lvcreate -L 20G -n lv_app vg_data      # 5. tạo LV 20 GB
mkfs.xfs /dev/vg_data/lv_app           # 6. tạo filesystem
mkdir -p /data && mount /dev/vg_data/lv_app /data
```

```
# Mở rộng thêm 10 GB, -r tự resize filesystem
lvextend -r -L +10G /dev/vg_data/lv_app
```

Mount cố định qua `/etc/fstab`

```
UUID=3f1c...  /data  xfs  defaults,nofail  0 0
```

- Dùng **UUID** (lấy bằng `blkid`) vì tên `/dev/sdX` có thể đổi sau khi khởi động lại
- `nofail`: disk lỗi thì máy vẫn boot được
- Luôn chạy `mount -a` để kiểm tra trước khi reboot

Lỗi thường gặp: sai một dòng trong fstab có thể khiến máy không boot, phải vào chế độ emergency để sửa.

🎯 **Đầu ra:** Tự thêm được disk mới, tạo LVM và mount vào hệ thống

TUẦN 1 · CHỦ ĐỀ 4

User, group và permission

User & group

- `/etc/passwd` : thông tin user
- `/etc/shadow` : mật khẩu đã hash
- `/etc/group` : nhóm và thành viên
- UID **0** = root · UID < 1000: tài khoản hệ thống
- Service account dùng shell `/sbin/nologin`

Quyền rwx

	File	Thư mục
r (4)	Đọc nội dung	Liệt kê file
w (2)	Sửa nội dung	Tạo/xóa file
x (1)	Chạy file	Đi vào (`cd`)

`750` = owner `rwx`, group `r-x`, others `---`

Quyền đặc biệt

- **SUID** (4): chạy với quyền owner, ví dụ `passwd`
- **SGID** (2) trên thư mục: file mới kế thừa group
- **Sticky** (1): chỉ owner xóa được file, ví dụ `/tmp`
- **umask** `022` → file mới 644, thư mục 755
- **ACL** (`setfacl`) khi cần cấp quyền cho từng user cụ thể

```
groupadd webapp
useradd -r -s /sbin/nologin -g webapp svc_web
chown -R svc_web:webapp /opt/webapp
chmod 750 /opt/webapp
chmod 640 /opt/webapp/config.env
setfacl -m u:deploy:rx /opt/webapp   # ACL cho user deploy
```

Nguyên tắc least privilege: mỗi ứng dụng chạy bằng user riêng, chỉ có quyền đúng thư mục cần dùng. Cấp sudo qua `visudo`, giới hạn đúng lệnh cần thiết, không cấp `ALL` khi không cần.

TUẦN 1 · CHỦ ĐỀ 5

Service với systemd

Khái niệm

- **systemd** là tiến trình PID 1, khởi động và quản lý mọi dịch vụ
- Đơn vị quản lý là **unit**: `.service`, `.socket`, `.timer` (thay cron), `.target` (nhóm unit)
- Unit file đặt trong `/etc/systemd/system/` (ưu tiên hơn `/usr/lib/systemd/system/`)
- **start** chạy ngay · **enable** tự chạy khi boot. Hai lệnh độc lập nhau

```
systemctl daemon-reload            # sau khi sửa unit file
systemctl enable --now webapp      # bật khi boot + chạy ngay
systemctl status webapp            # active / inactive / failed
systemctl reload webapp            # nạp lại config
journalctl -u webapp -p err --since "1 hour ago"
```

```
# /etc/systemd/system/webapp.service
[Unit]
Description=Web Application
After=network-online.target

[Service]
User=svc_web
WorkingDirectory=/opt/webapp
ExecStart=/opt/webapp/bin/server --port 8080
Restart=on-failure
RestartSec=5
MemoryMax=1G

[Install]
WantedBy=multi-user.target
```

Restart=on-failure giúp dịch vụ tự khởi động lại khi bị crash.

TUẦN 1 · CHỦ ĐỀ 5

Firewall: mở và đóng port theo yêu cầu

Nguyên tắc

- **Mặc định chặn**, chỉ mở port cần dùng
- Giới hạn nguồn truy cập khi có thể (ví dụ DB chỉ nhận từ VM App)
- Các công cụ firewalld, ufw đều cấu hình **netfilter** (nftables/iptables) trong kernel

firewalld (RHEL/Rocky)

- Quản lý theo **zone** (public, internal...)
- **runtime** mất khi reload · **–permanent** lưu cố định
- Mở theo **service** (`http`) hoặc theo **port** (`8080/tcp`)

Kiểm tra sau khi mở port

1. Dịch vụ đang lắng nghe? `ss -tulpn`
2. Firewall đã mở? `firewall-cmd --list-all`
3. Từ máy khác: `nc -zv <ip> 8080`

```
# firewalld
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-port=8080/tcp
firewall-cmd --permanent --remove-port=8080/tcp
# chỉ VM App (.12) được vào port DB 5432
firewall-cmd --permanent --zone=internal --add-source=192.168.10.
firewall-cmd --permanent --zone=internal --add-port=5432/tcp
firewall-cmd --reload
```

```
# ufw (Ubuntu)
ufw default deny incoming
ufw allow 22/tcp
ufw allow from 192.168.10.12 to any port 5432 proto tcp
ufw delete allow 8080/tcp
ufw status numbered
```

🎯 **Đầu ra:** Quản lý được vòng đời dịch vụ; mở/đóng port theo yêu cầu

TUẦN 1 · CHỦ ĐỀ 6

Hypervisor và ảo hóa

Type 1 – Bare-metal

Cài trực tiếp lên phần cứng, hiệu năng cao, dùng trong datacenter.

Ví dụ: VMware ESXi, Proxmox VE (KVM), Microsoft Hyper-V, XCP-ng

Type 2 – Hosted

Chạy như phần mềm trên một hệ điều hành có sẵn, phù hợp lab và học tập.

Ví dụ: VirtualBox, VMware Workstation

vCPU & RAM

- Hypervisor chia CPU/RAM vật lý thành vCPU/vRAM cho từng VM
- **Overcommit**: tổng vCPU có thể lớn hơn số core thật, cần theo dõi để tránh tranh chấp

Disk ảo

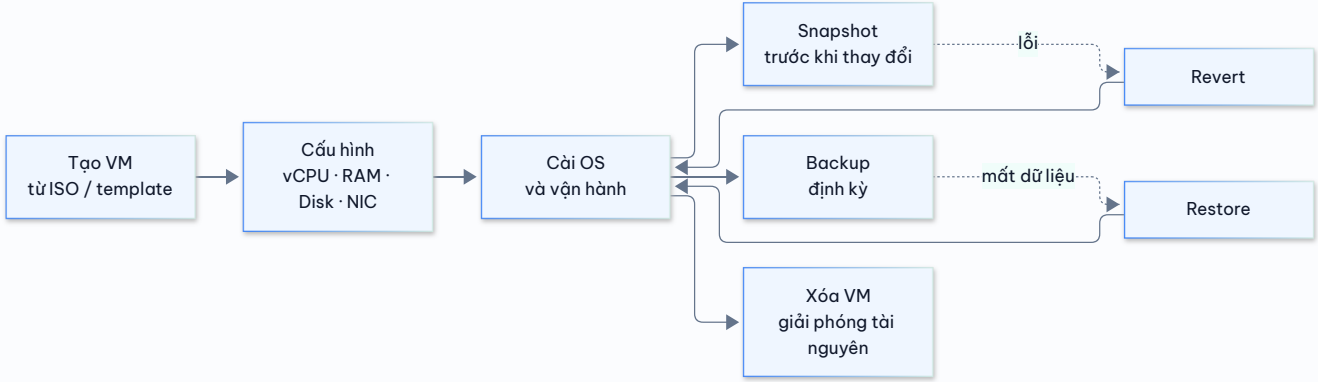
- **Thin**: cấp dần khi ghi, tiết kiệm dung lượng
- **Thick**: cấp đủ ngay, hiệu năng ổn định
- Định dạng: `qcow2` (KVM), `vmdk` (VMware), `vdi` (VirtualBox)

Mạng ảo

- **Virtual switch / bridge** nối các VM với nhau và với mạng vật lý
- VM có card mạng ảo (vNIC), có thể gắn nhiều mạng khác nhau

TUẦN 1 · CHỦ ĐỀ 6

Vòng đời VM: tạo, cấu hình, snapshot, backup



	Snapshot	Backup	Clone / Template
Bản chất	Trạng thái tại một thời điểm, phụ thuộc disk gốc	Bản sao độc lập, lưu nơi khác	Bản sao để tạo VM mới
Dùng khi	Trước khi update, đổi cấu hình	Khôi phục khi hỏng disk, mất dữ liệu	Triển khai nhanh nhiều VM giống nhau
Lưu ý	Không giữ lâu, làm chậm disk	Phải thử restore định kỳ	Đổi hostname, IP sau khi clone

Snapshot không thay thế backup: nếu disk gốc hỏng thì snapshot cũng mất theo.

Quy tắc 3-2-1: 3 bản dữ liệu, trên 2 loại phương tiện, 1 bản ở nơi khác.

Đầu ra: Thao tác được đầy đủ vòng đời VM trên hypervisor đang dùng

TUẦN 2

Container và Kubernetes Foundation

Image & Registry | Volume | Network | Resource limit | Kiến trúc Kubernetes

TUẦN 2 · CHỦ ĐỀ 7

Container hoạt động như thế nào?

Container là tiến trình được cô lập

Container không phải máy ảo. Đó là tiến trình chạy trên **kernel của host**, được kernel Linux cô lập bằng 3 cơ chế:

- **Namespaces**: cô lập những gì tiến trình *nhìn thấy*: `pid`, `net`, `mnt`, `uts` (hostname), `ipc`, `user`
- **cgroups**: giới hạn tài nguyên tiến trình *được dùng*: CPU, RAM, I/O
- **Union filesystem** (overlay2): ghép các layer chỉ đọc của image với một layer ghi riêng cho container

Tiêu chí	Virtual Machine	Container
Ảo hóa	Phần cứng (hypervisor)	Hệ điều hành (kernel)
Hệ điều hành	Mỗi VM một Guest OS	Dùng chung kernel host
Kích thước	Vài GB	Vài chục – vài trăm MB
Khởi động	Phút	Giây
Mức cô lập	Mạnh	Yếu hơn
Phù hợp	Nhiều OS, cô lập cao	Microservice, CI/CD

Tóm lại: VM ảo hóa cả một máy tính. Container chỉ cô lập ứng dụng, nên nhẹ và nhanh hơn nhưng phải dùng chung kernel.

🎯 **Đầu ra:** Giải thích được cơ chế container và khác biệt so với VM trong 5 phút

TUẦN 2 · CHỦ ĐỀ 7

Image và Registry

Image

- Template **chỉ đọc**, gồm nhiều **layer**. Mỗi lệnh trong Dockerfile tạo một layer
- Layer được **cache và dùng chung** giữa các image → build và pull nhanh
- Container = image + một layer ghi. Xóa container thì layer ghi mất

Registry

- Nơi lưu và phân phối image: Docker Hub, Harbor, GitLab Registry, ECR...
- Tên đầy đủ: `registry.company.vn/team/webapp:1.4.2`
- **Không dùng tag** `latest` cho production vì không biết chính xác phiên bản đang chạy

```
# Multi-stage build: image cuối nhỏ gọn
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci          # layer được cache
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
USER node           # không chạy bằng root
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

```
docker build -t registry.company.vn/team/webapp:1.4.2 .
docker push registry.company.vn/team/webapp:1.4.2
```

TUẦN 2 · CHỦ ĐỀ 7

Volume, network và resource limit

Volume

Dữ liệu trong container mất khi container bị xóa, nên dữ liệu cần giữ phải để ngoài:

- **Named volume**: Docker quản lý, dùng cho database
- **Bind mount**: gắn thư mục host, tiện khi dev
- **tmpfs**: nằm trên RAM, dữ liệu tạm

Network

- **bridge** (mặc định): mạng riêng trên host, ra ngoài qua NAT
- **user-defined bridge**: container gọi nhau **bằng tên** nhờ DNS nội bộ
- **host**: dùng chung mạng của host
- **none**: không có mạng
- `-p 8080:80`: publish port host 8080 → container 80

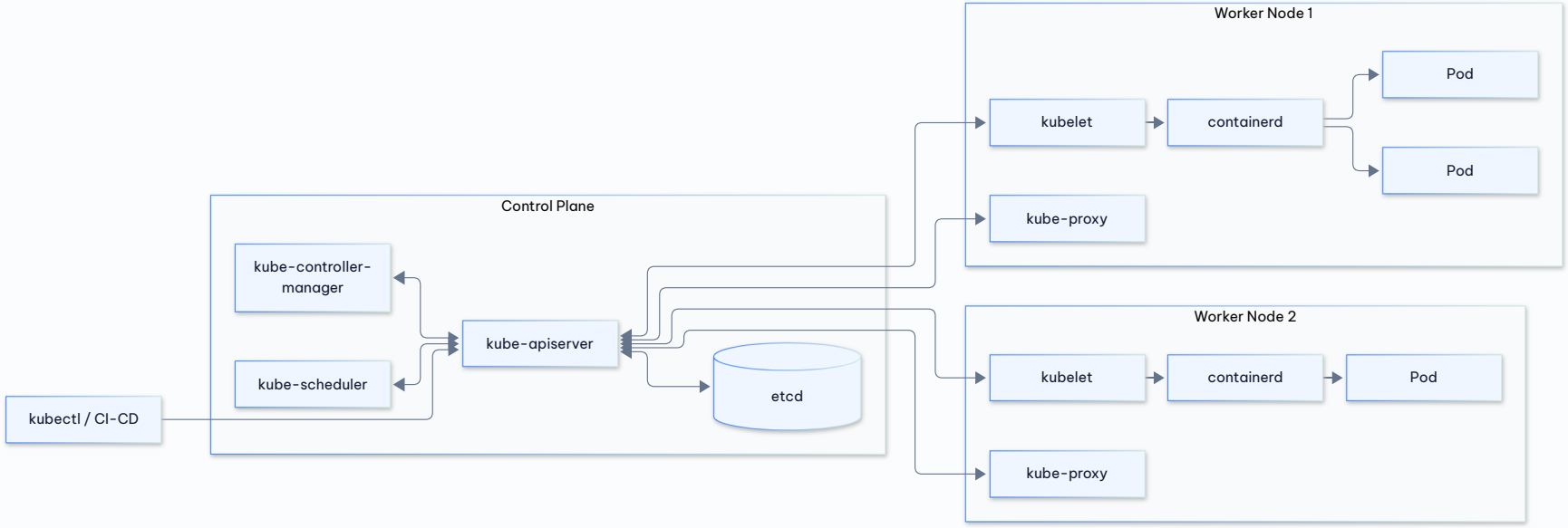
Resource limit

- `--memory 512m`: vượt giới hạn thì container bị **OOM kill**
- `--cpus 1.5`: tối đa 1,5 core
- `--pids-limit`: giới hạn số tiến trình
- Không đặt limit thì một container có thể chiếm hết tài nguyên của host

```
docker network create appnet
docker volume create pgdata
docker run -d --name db  --network appnet -v pgdata:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD_FILE=/run/secrets/db_pass --memory 1g --cpus 1 postgres:16
docker run -d --name web --network appnet -p 8080:3000 \
  -e DB_HOST=db --memory 512m --cpus 0.5 registry.company.vn/team/webapp:1.4.2
docker stats                    # theo dõi CPU/RAM thực tế của từng container
```

TUẦN 2 · CHỦ ĐỀ 8

Kiến trúc Kubernetes



Control Plane là "bộ não": nhận yêu cầu, lưu trạng thái mong muốn, quyết định Pod chạy ở đâu và liên tục sửa lệch.

Worker Node là nơi chạy workload: nhận lệnh từ apiserver, chạy container và cấu hình mạng cho Service.

TUẦN 2 · CHỦ ĐỀ 8

Vai trò từng thành phần

Control Plane	Vai trò
kube-apiserver	Cổng duy nhất vào cluster. Xác thực, phân quyền, ghi/đọc etcd. Mọi thành phần khác giao tiếp qua đây
etcd	CSDL key-value lưu toàn bộ trạng thái cluster. Mất etcd là mất cluster, nên phải backup
kube-scheduler	Chọn node cho Pod mới dựa trên tài nguyên còn trống, affinity, taint/toleration
controller-manager	Chạy các controller (Deployment, ReplicaSet, Node...), so sánh trạng thái thực với mong muốn rồi sửa

Worker Node	Vai trò
kubelet	Agent trên mỗi node. Nhận Pod được giao, gọi runtime chạy container, báo cáo trạng thái và health check
container runtime	Chạy container thật qua chuẩn CRI: containerd, CRI-O
kube-proxy	Cấu hình iptables/IPVS để traffic tới **Service** được chia đều về các Pod
CNI plugin	Cấp IP cho Pod, cho Pod ở các node khác nhau nói chuyện được: Calico, Flannel, Cilium

Đối tượng cơ bản: Pod (đơn vị nhỏ nhất, 1+ container) · **Deployment** (quản lý số bản sao, rolling update) · **Service** (IP/DNS ổn định trước nhóm Pod) · **Namespace** (chia cluster theo team/môi trường) · **ConfigMap / Secret** (cấu hình và thông tin nhạy cảm)

TUẦN 2 · CHỦ ĐỀ 8

Luồng xử lý khi triển khai một ứng dụng

1. **kubectl apply** gửi Deployment (replicas: 3) tới **kube-apiserver**; apiserver xác thực rồi lưu vào **etcd**
2. **Deployment controller** thấy Deployment mới → tạo ReplicaSet → ReplicaSet tạo **3 Pod** (chưa có node)
3. **kube-scheduler** thấy Pod chưa có node → chọn node phù hợp và ghi lại qua apiserver
4. **kubelet** trên node đó thấy Pod được giao → gọi **containerd** pull image và chạy container
5. kubelet báo trạng thái **Running** về apiserver; **kube-proxy** cập nhật routing nếu có Service trỏ tới Pod

Cơ chế cốt lõi: khai báo trạng thái mong muốn, hệ thống tự điều chỉnh. Pod chết thì controller tạo Pod mới thay thế.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: webapp }
spec:
  replicas: 3
  selector:
    matchLabels: { app: webapp }
  template:
    metadata:
      labels: { app: webapp }
    spec:
      containers:
      - name: web
        image: team/webapp:1.4.2
        resources:
          limits: { cpu: 500m, memory: 512Mi }
```

🎯 **Đầu ra:** Vẽ lại được kiến trúc Kubernetes không nhìn tài liệu

TỔNG KẾT

Kết quả đạt được sau 2 tuần

#	Mảng kiến thức	Đầu ra theo lộ trình	Trạng thái
1	Linux: tài nguyên máy chủ	Kiểm tra và phân bổ được tài nguyên; giải thích được cho người khác	✅
2	Network: IP, routing, DNS	Vẽ và giải thích được luồng kết nối mạng giữa các VM trong Lab	✅
3	Storage: partition, LVM, mount	Tự thêm được disk mới, tạo LVM và mount vào hệ thống	✅
4	User, group, permission	Tạo và phân quyền được theo đúng yêu cầu của ứng dụng	✅
5	systemd và firewall	Quản lý được vòng đời dịch vụ; mở/đóng port theo yêu cầu	✅
6	Hypervisor và vòng đời VM	Thao tác được đầy đủ vòng đời VM trên hypervisor đang dùng	✅
7	Container	Giải thích được cơ chế container và khác biệt so với VM trong 5 phút	✅
8	Kiến trúc Kubernetes	Vẽ lại được kiến trúc Kubernetes không nhìn tài liệu	✅

Điểm mạnh: .....

Cần củng cố thêm: .....

Cảm ơn anh/chị đã lắng nghe

Kế hoạch tiếp theo: .....

Hỏi & Đáp